Politecnico di Milano

Ingegneria del Software – a.a. 2008/09

Appello del 8 Luglio 2008

Cognome

Nome

Matricola

Sezione (segnarne una) Baresi □, Ghezzi □, SanPietro □

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 12)
La classe **Magazzino** consente di gestire un semplice magazzino per lo stoccaggio delle merci con 1000 slot. Non interessa sapere quali merci vengono immagazzinate, ma solo la quantità totale di ciascun tipo di merce. Si ipotizzi per semplicità che la quantità sia misurabili in unità; essa non deve mai eccedere la capacità massima. La classe mette a disposizione i seguenti metodi observer:

quantita(int cod), restituisce il numero di unità della merce di codice cod presenti in magazzino
ArrayList<Integer> merci(), restituisce tutti e soli i codici di tutte le merci stoccate (senza restituirne la quantità)

domanda a
Fornire la specifica JML dei metodi seguenti, che rilanciano eccezioni nel caso di superamento dei limiti di capacità.

```
//@ requires qta >= 0;

//@ ensures
// precondizione spostata come postcondizione

//@ (\sum int i; i<=0 && i<\old(merci()).size(); \old(quantita(merci().get(i)))) <=1000-qta &&

// Tutti i vecchi codici devono continuare ad esistere e se sono diversi da cod anche la quantita
// deve restare uguale

//@ (\forall int i; 0 <= i && i < \old(merci()).size();
//@    (\exists int j; 0 <= j && j < merci().size(); \old(merci()).get(i) == merci().get(j) &&
//@       merci().get(j) != cod ==> \old(quantita(merci().get(j))) == quantita(merci().get(j)))) &&

// La quantita di merce di codice cod deve essere incrementata di qta

//@ quantita(cod) == \old(quantita(cod)) + qta &&

// Se il cod non era presente, la lunghezza di merci deve essere incrementata di uno; altrimenti
// resta uguale. Questo dovrebbe bastare se facciamo l'ipotesi ovvia che i codici non siano
// ripetuti nella lista

//@ \old(merci()).contains(cod)? merci().size() == \old(merci()).size:
//@                              merci().size() == \old(merci()).size + 1;

//@ signals (CapacitaRaggiuntaException e)
//@ (\sum int i; i<=0 && i< \old(merci()).size(); \old(quantita(merci().get(i)))) > 1000-qta;

public void aggiungi(int cod, int qta) throws CapacitaRaggiuntaException

//@ requires qta >= 0;

//@ ensures
// precondizione spostata come postcondizione

//@ \old(quantita(cod))>= qta &&

// Tutti i nuovi codici devono continuare ad esistere e se sono diversi da cod anche la quantita
// deve restare uguale

//@ (\forall int i; 0 <= i && i < merci().size();
//@    merci().get(j) != cod ==> (\exists int j; 0 <= j && j < \old(merci()).size();
//@             merci().get(i) == \old(merci()).get(j) &&
```

```
//@                \old(quantita(merci().get(j))) == quantita(merci().get(j))) &&

// La quantita di merce di codice cod deve essere decrementata di qta

//@ quantita(cod) == \old(quantita(cod)) - qta &&

// Se la quantita di cod diventa zero, la lunghezza di merci deve essere decrementata di uno;
// altrimenti resta uguale. Questo dovrebbe bastare se facciamo l'ipotesi ovvia che i codici
// non siano ripetuti nella lista

//@ quantita(cod)==0? merci().size() == \old(merci()).size-1:
//@                   merci().size() == \old(merci()).size;
```

public void preleva(int cod, int qta) throws merceInsufficienteException

domanda b

La classe **MagazzinoVirtuale** fornisce le stesse operazioni di **Magazzino**, ma senza limite di stoccaggio. Dopo aver specificato in JML il metodo **aggiungi(int cod, int qta)**, opportunamente definito, spiegare brevemente e chiaramente se è possibile definire **MagazzinoVirtuale** come estensione di **Magazzino**, rispettando il principio di sostituzione.

```
//@ requires qta >= 0;

//@ ensures

// incremento di quantita della sola merce con codice cod

//@ (\forall int i; i<=0 && i<merci().size());
//@    merci().get(i) == cod? quantita(merci().get(i)) == \old(quantita(cod))+qta :
//@                           quantita(merci().get(i)) == \old(quantita(merci().get(i))) &&

// eventualmente aggiungo solo cod alla lista di codici

//@ !(\exists int j; j<=0 && j <\old(merci()).size());
//@    \old(merci()).get(j)==merci().get(i)) ==> merci().get(i)==cod) &&

// e lo faccio una volta sola

//@ merci().size()-\old(merci()).size() <= 1;
```

public void aggiungi(int cod, int qta) throws CapacitaRaggiuntaException

Non è possibile rispettare il principio di sostituzione.
La regola delle segnature sarebbe in realtà verificata, in quanto tutti i metodi di MagazzinoVirtuale hanno la stessa segnatura che in Magazzino, salvo l'assenza dell'eccezione CapacitaRaggiuntaException.
Tuttavia, la postcondizione completa del metodo aggiungi() include anche la parte signals, ossia il lancio dell'eccezione al raggiungimento di 1000 unità. Formalmente, la postcondizione di Magazzino.aggiungi è del tipo:

(CapacitaOK&& aggiornaQuantita && nonLanciareCapacitaRaggiuntaException) ||
(!!CapacitaOK && quantitaInvariata && lancia CapacitaRaggiuntaException)

La postcondizione di MagazzinoVirtuale.aggiungi è invece del tipo:

aggiornaQuantita && nonLanciareCapacitaRaggiuntaException

Ma questa condizione non implica la postcondizione precedente (in quanto ci sono casi in cui CapacitaOK è falso). Pertanto, si viola la regola dei metodi.

Ad esempio, la nuova classe "sorprenderebbe" il codice che si aspettasse di inserire 1000 oggetti per poi ricevere un'eccezione. Ad esempio, il codice seguente inserisce 1000 oggetti in magazzino.

```
public void riempi(Magazzino m,int cod) {
try {
        for (;;) m.aggiungi(cod,1);
catch { CapacitaRaggiuntaException e}
}
```

Se chiamata su un oggetto di tipo MagazzinoVirtuale, il codice risulterebbe in un loop infinito.

domanda c

c1- Si consideri un'altra versione della classe **Magazzino**, in cui la specifica del metodo **aggiungi** non prevede il lancio dell'eccezione **CapacitaRaggiuntaException**: il superamento del limite e' considerato un prerequisito sul chiamante. Scrivere la specifica del metodo **aggiungi** in questa nuova versione:

```
//@ requires qta >= 0 &&
//@ (\sum int i; i<=0 && i<\old(merci()).size(); \old(quantita(merci().get(i)))) <=1000-qta

//@ ensures
// incremento di quantita della sola merce con codice cod

//@ (\forall int i; i<=0 && i<merci().size());
//@     merci().get(i) == cod? quantita(merci().get(i)) == \old(quantita(cod))+qta :
//@                            quantita(merci().get(i)) == \old(quantita(merci().get(i))) &&

// eventualmente aggiungo solo cod alla lista di codici

//@ !(\exists int j; j<=0 && j <\old(merci()).size());
//@     \old(merci()).get(j)==merci().get(i)) ==> merci().get(i)==cod) &&

// e lo faccio una volta sola

//@ merci().size()-\old(merci()).size() <= 1;
```

public void aggiungi(int cod, int qta)

c2- Si consideri ora la stessa classe MagazzinoVirtuale specificata al punto (b). Se questa fosse definita come estensione di Magazzino, il principio di sostituzione sarebbe verificato? Motivare la risposta.

Sì, in quanto il metodo aggiungi di MagazzinoVirtuale indebolisce la precondizione, mantendo invariata la postcondizione.

Esercizio 2 (punti 7)
Si consideri il seguente metodo statico

public static boolean triangolare (int[][] m, char t)

che presa in ingresso una matrice quadrata, restituisce true se t vale 's' e la matrice è una matrice triangolare superiore (ovvero, tutti gli elementi al di sotto della diagonale principale sono uguali a zero), oppure se t vale 'i' e la matrice è una matrice triangolare inferiore (tutti gli elementi al di sopra della diagonale principale sono uguali a zero).

Si definisca la specifica del metodo statico senza considerare possibili eccezioni

```
//@ requires
// matrice non nulla o vuota:

//@ m != null && m.length > 0

// matrice quadrata senza righe nulle:

//@ (\forall int i; i<= 0 && i< m.length; m[i] != null && m[i].length==m.length) &&

// carattere corretto:

//@ t == 's' || t == 'i';


//@ ensures

//@ t == 's' &&
                (\forall int i; 0<i && i< m.size();
                        (\forall int j; 0<=j && j< i; m[i][j]==0))
        || t == 'i' &&
                (\forall int i; 0<=i && i< m.size()-1;
                        (\forall int j; i<j && j< m.size(); m[i][j]==0));
// la matrice non e' modificata al termine:
//@assignable nothing;
```

Si aggiungano le relative eccezioni, e si modifichino le pre- e post-condizioni ove necessario, per rendere il metodo totale.

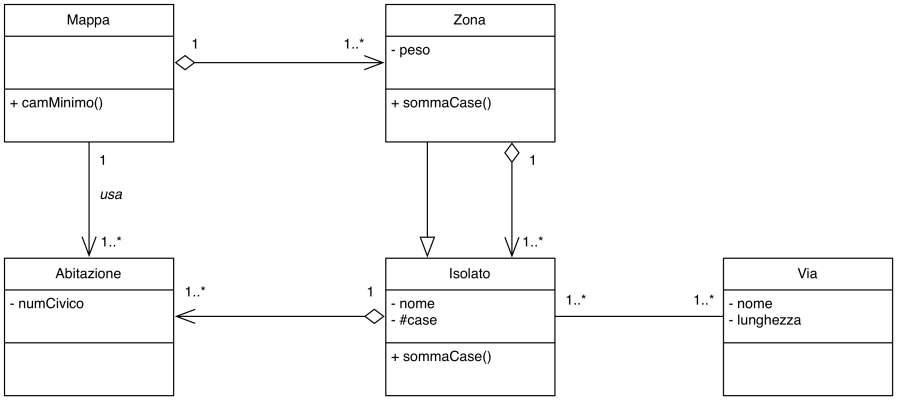
Si dichiari che il metodo può lanciare le eccezioni NullPointerException, EmptyException, NonQuadrataException, CarattereException si tolga la requires e si aggiungano:

m!=null && (\forall int i; 0<=i && i< m.size(); m[i].length == m.length) && (t == 's' || t == 'i')
in AND alla ensures, e inoltre le signals:

```
signals (NullPointerException e) m==null || (\exists int i; 0<=i && i< m.length; m[i]==null);
signals (EmptyException e) m.length == 0;
signals (NonQuadrataException e) m!=null && !(\forall int i; 0<=i && i< m.length; m[i].length ==
m.length);
signals ( CarattereException e) !(t == 's' || t == 'i')
```

Esercizio 3 (punti 6)
La mappa di una città può essere vista a diversi livelli gerarchici. In prima approssimazione possiamo pensare che sia un insieme di zone connesse da vie principali. Ogni zona ha un nome, un peso e al proprio interno può contenere altre zone e/o isolati. Ogni via ha un nome e una lunghezza, connette due o più zone, ma può anche collegare (separare) due o più isolati. Ogni isolato è identificato da un nome e dal numero di case (numeri civici) in esso contenute. Una via può solo collegare isolati della stessa zona, oppure zone diverse, ma non isolati di zone diverse.
Ogni zona deve poter conoscere la somma delle case in essa contenute, direttamente o transitivamente. La mappa deve essere in grado di calcolare il cammino minimo (rispetto alla somma delle lunghezze delle vie attraversate) tra due abitazioni di due zone qualsiasi.

domanda a
Descrivere con un diagramma delle classi UML la specifica sopra riportata, aggiungendo anche tutti e soli gli attributi e metodi rilevanti per la specifica.



domanda b
Scrivere la specifica JML del modo che calcola il numero di case in una zona, in funzione del modello UML definito al punto precedente.

Supponendo di usare un array zone per implementare l'aggregazione tra Zona e Isolato, la postcondizione sarebbe:

//@ ensures \result == (\sum int i; i>=0 && i< zone.length; zone[i].sommaCase());